The Impact of Hyperparameters on Large Language Model Inference Performance: An Evaluation of vLLM and HuggingFace Pipelines

Matias Martinez
Universitat Politècnica de Catalunya (UPC) - BarcelonaTech
Barcelona, Spain
matias.martinez@upc.edu

arXiv:2408.01050v1 [cs.SE] 2 Aug 2024

***Abstract*—The recent surge of open-source large language models (LLMs) enables developers to create AI-based solutions while maintaining control over aspects such as privacy and compliance, thereby providing governance and ownership of the model deployment process. To utilize these LLMs, inference engines are needed. These engines load the model's weights onto available resources, such as GPUs, and process queries to generate responses. The speed of inference, or performance, of the LLM, is critical for real-time applications, as it computes millions or billions of floating point operations per inference. Recently, advanced inference engines such as `vLLM` have emerged, incorporating novel mechanisms such as efficient memory management to achieve state-of-the-art performance. In this paper, we analyze the performance, particularly the throughput (tokens generated per unit of time), of 20 LLMs using two inference libraries: `vLLM` and HuggingFace's *pipelines*. We investigate how various hyperparameters, which developers must configure, influence inference performance. Our results reveal that throughput landscapes are irregular, with distinct peaks, highlighting the importance of hyperparameter optimization to achieve maximum performance. We also show that applying hyperparameter optimization when upgrading or downgrading the GPU model used for inference can improve throughput from HuggingFace pipelines by an average of 9.16% and 13.7%, respectively.**

I. Introduction

Large language models (LLM) have revolutionized the manner in which developers build *intelligent* software solutions, affecting several areas, including software engineering ([52], [54], [25], [18]). One of the primary drivers of this revolution is the emergence of *open-source* LLMs, which offer publicly accessible architecture, code, checkpoints (model weights) and, eventually, training data. Open-source LLMs enables developers to create AI-based solutions while maintaining control over aspects such as privacy and compliance, and, at the same type, providing governance and ownership of the model deployment process. Platforms such as HuggingFace (HF) have contributed to the success of open-source AI by providing, for instance, more than 700.000 models for download, including LLMs such as CodeLlama [39].

Inference engines (or frameworks) are software components that make LLMs operable. They have as main responsibility to load the model's weighs into the available devices (GPU and/or CPU), and to make the *inference* i.e., to generate a prediction/output/response based on input data. HuggingFace, for instance, provides inference mechanisms in its `Transformer` library [48], such as the `pipelines`, which are objects that abstract complex code used for inference (by wrapping other libraries such as PyTorch) and offer a simple API dedicated to several tasks, including text generation.

Listing 1: HuggingFace pipeline

```
model = AutoModelForCausalLM.from_pretrained(model_name, device_map='auto',[...])
return pipeline("text-generation",
    model=model,
    max_new_tokens=100,
    model_kwargs={"torch_dtype": torch.bfloat16} [...] )
```

A key factor for the success of an inference engine is *performance*, i.e., the speed of serving, because it directly impacts the user experience of AI-powered systems and on the cost (since LLMs require specialized hardware such as GPUs). A measure of LLM performance is *throughput*, expressed in tokens generated by an LLM per unit of time. Increasing the throughput of LLM inference, and therefore reducing the cost per request, is becoming primordial [27].

Recently, state-of-the-art inference engines have emerged with the goal of improving inference performance. One of them is `vLLM` [27], launched in June 2023, which implements a memory-efficient inference engine for LLM. Inference engines such as vLLM or these from HuggingFace can be integrated into applications with a few lines of code, as Listing 1 shows.

However, the inference engine has hyperparameters that developers need to set by the developers. For example, Listing 1 shows a real utilization of HuggingFace pipelines[1], which first loads the model (it has one hyperparameter `device_map`) and then creates a pipeline with the model as a parameter and some hyperparameters (e.g, `max_new_tokens`). Listing 2 shows real code (but simplified) that uses `vLLM`[2]. The first line loads the model (with hyperparameter `tensor_parallel_size`), the second performs inferences from a list of prompts (receives two hyperparameter `temperature` and `top_p`).

There are hyperparameters that have a direct incidence

[1]https://github.com/adit-copilot/EnchantedQuest/blob/48d6256c482abc40a8c916c6c8d1a7c5edb2c6fa/retrival/qa_pipeline.py#L46

[2]https://github.com/tencent-ailab/persona-hub/blob/5e9b7d0a29eff2e2eaf01cc8c74d4abb731fb4a0/code/vllm_synthesize.py#L37

Listing 2: vLLM

```
llm = LLM(model=model_path, tensor_parallel_size=4)
#tensor_parallel_size based on the GPUs you are using
outputs = llm.generate(prompts, SamplingParams(temperature=0.6, top_p=0.95, [...]))
```

in throughput while others such as `temperature` direct affects the output from an LLM. In this paper, we focus on the former. One of them is `tensor_parallel_size` from `vLLM`, which indicates the number of GPUs to use for distributed execution with tensor parallelism. The value of this hyperparameter needs to be set by the developer according to the resources available in the deployment infrastructure. In the code from Listing 2, `tensor_parallel_size` is hard-coded to 4 and includes the comment: *"#tensor_parallel_size based on the GPUs you are using"*.

Another hyperparameter that affects throughput is *batch size*, which refers to the number of input instances processed simultaneously by the model in a single inference call. According to [50], the batch size is one of the most important hyperparameters to tune during the training phase, affecting, for example, accuracy and training time. In inference, increasing batch size typically increases throughput [27]. However, determining the batch size is challenging as is subject to limitations of the memory capacity of the hardware (e.g., it is not able to allocate the memory for numerous inputs on a large batch). On the other hand, a low value can lead to hardware being underutilized.

In this paper, we study how hyperparameters of inference engines affect the throughput of LLM. In particular, we examine the throughput landscape defined by the spaces of two hyperparameters, the number of GPUs to be used, and the batch size. We study LLM inference in the context of the code completion task, which is implemented by products such as Microsoft GitHub Copilot [56], [57]. As noted by [47], copilot efficiency is essential because LLM inference is costly and unsustainable on a massive scale. In our experiment, we study 20 open-source LLM, all from key players in the AI industry such as Meta, Google, Microsoft, Mistral, and HuggingFace, and two popular libraries that enable inference: HuggingFace Transformers [48] and `vLLM` [27].

Our results reveal that the throughput landscapes are irregular, with distinct peaks, underscoring the importance of hyperparameter optimization for maximizing inference performance. Motivated by these findings, we conduct an experiment on hyperparameter optimization. We examined two use cases focusing on changes in the inference infrastructure, such as upgrading or downgrading the GPU model. Our experiment shows that adjusting the number of GPU hyperparameters and/or batch size using our tool `InfPop` (based on Hyperopt [5]) in Hugging Face inference improves throughput by an average of 9.16% when upgrading the GPU model (from Nvidia V100 to A100) and by 13.7% when downgrading (from Nvidia A100 to V100).

All data, plots, and scripts are available in our Appendix [1]. The paper continues as follows. Section II presents the research question we investigate. Section III describes the research protocol. Section IV presents the results. Section V presents the discussion. Section VI discusses related work. Section VII presents the conclusions and future work.

II. Research questions

The research questions that guide this experiment are the following:

RQ1: What is the shape of the throughput landscape across the hyperparameter space defined by the batch size and the number of GPUs on `vLLM` inference?

For each LLM under study, we visually inspect the throughput distribution within the hyperparameter space defined by the number of GPUs and the batch size to analyze its shape. In particular, we search for: peaks, flat regions, or valleys.

This analysis gives us the first sing to know whether hyperparameter optimization should be applied to maximize or minimize a metric, such as throughput. For example, a peak in the landscape is a sign that there is a set of hyperparameters that maximize throughput.

RQ2: For online inference, how does throughput vary with different numbers of GPUs (hardware scaling)?

The purpose of this question is to study the impact of the number of GPUs on throughput. This question could help us to know, for example, whether having more GPUs in the inference infrastructure (called *GPU scaling*) allows an inference engine to improve throughput.

In this research question, we focus on *online inference*, each query done to the model (i.e., a request) through the inference library includes just one input. As a result, the model generates one output (or eventually more than one if applying, for example, beam search) for that single input.

RQ3: For batched inference, how does throughput vary with the batch size?

This research question focuses on the *batch* inference. In contrast to the previous one, where the query to the LLM consists of one *single* input, the batching considers n inputs in a single query. This n is the *batch size*, which is an hyperparameter of the inference processes. Choosing an appropriate batch size n is challenging. On the one hand, a high batch size value would lead to throughput improvements but also may lead to out-of-memory. On the other hand, a low value may lead to resources (e.g., GPUs) being underutilized. Here, we study the relation between the space of the hyperparameter batch size and the throughput from inference engines.

RQ4: How does throughput vary between different GPU models during inference?

Each new generation of GPUs boasts improved performance including throughput [8]. In this research question, we measure the variation in throughput across two GPU models, one representing a recent and widely used model, and another model from a previous generation.

RQ5: To what extent does applying Hyperparameter Optimization improve throughput in LLM?

The goal is to present an use case in which hyperparameter optimization of LLM inference produces improvements in throughput. In particular, we apply hyperparameter optimization (using our approach `InfPop`) in cases where there are changes on the hardware infrastructure used for inference. Thus, hyperparameter optimization finds a new hyperparameter values that maximize throughput on the new hardware.

III. Experimental protocol

In order to answer the research questions, we conduct an experiment that consists of executing the code-completion task for a set of incomplete programs. That completion is done by doing inference calls to a LLM using an inference engine. The experiment is designed as follows.

A. Selection of the inference engines to evaluate

We evaluate inference using two inference engines.

1) Inference from HuggingFace Transformers library: We select it for the following reasons. First, it offers simple, easy to use, and concise APIs to perform inference on LLM. For example, *pipelines* abstracts most of the complex code from the library, and provides a simple API dedicated to several tasks, including Text Generation. An example for this pipeline is shown in Listing 1. Internally, that code uses a pipeline `TextGenerationPipeline`. In the remainder of the paper, we will refer to the HuggingFace pipeline for text generation it as $\texttt{HF}_{pl}$. Secondly, HuggingFace provides thousads of models which can be used with $\texttt{HF}_{pl}$ for inference. Lastly, HF Transformers has is a relevant library in the the scene of open-source on AI: it has ≈130.000 stars on GitHub.

2) `vLLM`: originaly materialized to implement *PagedAttention* [27] a technique for efficient management of attention key and value memory of transformer architecture. We choose to study `vLLM` for different reasons, including: *a)* It provides state-of-the-art inference performance on LLM [27]. *b)* introduces a novel memory management (PagedAttention [27]), then integrated to other inference engines [28] such as TensorRT-LLM [38]. *c)* It integrates with HuggingFace models, enabling it to work with state-of-the-art LLMs. *d)* Popularity: more than 23.000 Github stars since its launch (June 2023). *e)* Integrated with other LLM libraries and frameworks such as LangChain[3].

B. Selection criteria of the LLM to evaluate

We select a model if all the following criteria are met.

- Model based on the Transformer architecture [45], which is effective for code completion tasks [41].
- As our study focuses on large language models (LLM), the model must have ≈>1 billion parameters.
- Evaluated on HumanEval [7] to demonstrate their competence in the code generation task.

[3]https://python.langchain.com/v0.2/docs/integrations/llms/vllm/

HuggingFace model id	#Params (Billion)	HumanEval (pass@1)
bigscience/bloom-1b7 [49]	1.7	4.4
bigscience/bloom-3 [49]	3	6.3
bigscience/bloom-7b1 [49]	7.1	8.1
facebook/CodeLlama-7b [39]	7	38.4
facebook/CodeLlama-13b [39]	13	43.3
facebook/CodeLlama-34b [39]	34	53.7
facebook/CodeLlama-70b	70	57.3
mistralai/Codestral-22B	22	81.1
mistralai/Mistral-7B-v0.1 [23]	7	30.5
mistralai/Mixtral-8x7B-v0.1 [24]	46.7	40.2
gemma-ai/gemma-2b [43]	2	22.0
gemma-ai/gemma-7b [43]	7	32.3
microsoft/phi-1 [20]	1.3	50.6
microsoft/phi-1.5 [30]	1.3	41.4
microsoft/phi-2 [22]	2.7	49.4
bigcode/starcoder-15b [29]	15.5	28.7
bigcode/starcoder2-3b [31]	3	31.7
bigcode/starcoder2-7b [31]	7	46.3
WizardLMTeam/WizardCoder-15B [32]	15	59.8
WizardLMTeam/WizardCoder-33B [32]	33	79.9

TABLE I: LLMs considered in this study.

- Model's weights (checkpoints) are available on the HuggingFace hub[4].
- Model's architecture implemented in the `HuggingFace Transformers` library[5] [48].
- Model tagged with "Text-Generation" tag[6] in the HuggingFace site for ensuring usage with HF pipeline.
- Model supported by the vLLM library.

To select the models to study, we first inspect the vLLM documentation and its Github repository to detect the supported models. As `vLLM` uses HuggingFace Transformers library, this strategy allows us to select models supported by both the $\texttt{HF}_{pl}$ and `vLLM`. For each supported model, we read its documentation, related paper, HuggingFace model card, and the dashboard Paper-on-Code[7] to check all mentioned criteria.

Table I shows the 20 LLM that meet the criteria mentioned above and work in our infrastructure. It shows the numbers of parameters, which range from 1.3 to 70 billions, and the HumanEval (pass@1) reported by the models's authors. We include LLM from key players in the AI industry, such as Meta, Google, Microsoft, Mistral, and HuggingFace[8] ([29], [31]). Other models are discarded because they do not meet some of the criteria. For example, Moisaic [44] has not been evaluated in HumanEval; CodeGen [37] and Incoder [13] are not supported by vLLM (last check June 2024). Microsoft-Phi-3 was later discarded because it failed in our infrastructure.

C. Evaluation benchmark

HumanEval [7] is a dataset of 164 handwritten programming problems proposed by OpenAI to evaluate functional

[4]HuggingFace model hub: https://huggingface.co/models

[5]Model architectures implemented in HuggingFace transformer library: https://github.com/huggingface/transformers/tree/14ff5dd962c1bd0a4e3adaac347ba396d8df5add/src/transformers/models and https://huggingface.co/docs/transformers/

[6]https://huggingface.co/models?pipeline_tag=text-generation

[7]https://paperswithcode.com/sota/code-generation-on-humaneval

[8]HuggingFace participates via BigCode: https://www.bigcode-project.org/

correctness and measure the problem-solving capabilities of their code-based models. We consider HumanEval as it has been used by big AI players (e.g., OpenAI, Meta, Google, Microsoft) to evaluate the capaibilities of their LLMs.

Researchers created new benchmarks from HumanEval, to evaluate other tasks beyond function generation. For example, Fried et al. [13] and Bavarian et al. [4] evaluated code *infiling* task, which consists of predicting missing text spans that are consistent with the preceding and subsequent code.

To evaluate the code completion task, we select *Single-line infilling*[9] [13]. They created it by marking out each non-blank line of code in the canonical function implementation from each HumanEval problem. In total, *Single-line infilling* contains 1033 examples. We use this bechmark in a slightly different manner: we query a model to generate code just based on the preceding code, and we ignore the subsequent code as code completion is not infilling.

D. Hardware used

We use two high-performance computation nodes available in our host infrastructure: two CPU Intel Xeon E5-2698 and 512 GB of RAM. The first one is equipped with eight Nvidia A100 GPUs with 40 GB [8] . We choose this node because that GPU has been extensively used to train and evaluate state-of-the-art LLMs considered in our study. The second node is equipped with a GPU model from a previous generation: Nvidia V100[10], which is the first Tensor Core GPU introduced by Nvidia designed specifically for deep learning. The node is equipped with eight V100 with 32 GB. We choose to consider V100 as a "previous generation" GPU because it has been used to train and evaluate some considered LLMs (e.g. Bloom [49]).

E. Executing Inference trials on the Single-line infilling

We call a *trial* to the execution of the code completion tasks for all items from the *Single-line infilling* benchmark by querying one LLM m, using an inference engine e ($e \in [\texttt{HF}_{pl}, \texttt{vLLM}]$), executed on n GPUs ($n \in [1, 2, 4, 8]$) of model g ($g \in [A100, V100]$), with a batch size b ($b \in [1, 2, 4, 8, 16, 32, 64, 128, 256, 1028]$). We run *trials* resulting from all the combinations between m, e, n, g and b. Note that, for each *trial* with batch size b, we create x inference invocations, each with b elements from the benchmark (x results from the division between the size of the benchmark and b). In `vLLM`, we modify the parameter of the constructor of class `LLM`. `tensor_parallel_size` for pasing the number of GPUs to be used, and pass b queries to the method `generate` from that class, where b is the batch size of the trial. In $\texttt{HF}_{pl}$, we keep `device_map="auto"` from the pipeline and, at the same time, we restringe the visible GPUs by changing the evironmental variable `CUDA_VISIBLE_DEVICES`. For example, for a trial that uses 2 GPUs, we set the value `0,1`. Then, the other GPUs ($2 \ldots 8$) are not visible. This strategy allows the pipeline, thanks to the `auto` configuration provides by the HF Accelerate library, to determine automatically the model deployment on the available (and visible) devices. For the batch size, we pass the value b via hyperparameter `batch_size` from the pipeline.

We keep the default values for the other hyperparameters from the inference engine, except for two. First, we indicate the inference engine to generate up to 5 tokens. Second, we unify the floating-point precision used. All models from Table I use by default half-precision floating point (FP16/float16) with the exception of one (bigcode/starcoder2-3b) that uses full-precision floating point (FP32/float32). To ensure a fair comparison between models and engines, we explicitly set the hyperparameter precision to FP16 in both inference engines as is the value from the large majority. For each trial, we calculate throughput as the number of tokens generated during the trial (composed by x inference invocations) divided by the execution time of the trial in seconds (t/s).

F. Data analysis to answer the research questions

Once executed all trials, we apply the following analysis to answer each research question.

1) RQ1: We plot the throughput landscapes using the Python library matplotlib. There is a plot for each combination of: *a)* hardware, *b)* inference engine, and *c)* LLM. We then inspect the shape of all the landscapes to select the representative cases for discussion.

2) RQ2: We select the trials with batch size equal to one. Then, for each LLM, we plot the throughput evolution with different numbers of GPUs, and report the number of GPUs that maximize throughput. We test the hypothesis H_0*: there is no statistically significant difference in throughput across different numbers of GPUs* by appling an analysis of variance (ANOVA). In this test, each group corresponds to the trial with the same number of GPUs used on inference (1, 2, 4 and 8). Finally, we compare throughput between the two inference engines, `vLLM` and $\texttt{HF}_{pl}$.

3) RQ3: For each combination of *a)* inference engine, *b)* GPU model, *c)* number of GPUs, and *d)* LLM, we plot the evolution throughput with the batch size. Then, for each combination, we test the null hypothesis H_0*: There is no a significant difference in the throughput values based on the batch size* using Analysis of Variance test (ANOVA)

4) RQ4: To compare the throughput obtained with a Nvidia A100 with that one obtained with a Nvidia V100, we first compare the throughput from pairs of trials, one from A100, the other from V100, but sharing the other hyperparameters (LLM, batch size, number of GPUs). Moreover, we compute, for each inference engine, the Wilcoxon Signed-Rank Test to test the null hypothesis H_0*: The difference in throughput between Nvidia A100 and v100 is not significant*.

5) RQ5: We evaluate the impact of hyperparameter optimization in two experiments: *1)* Upgrading and *2)* Downgrading GPU. Both consider two GPUs models, *previous* (P) and *new* (N). For studying the *Upgrading*, P becomes the

[9]https://github.com/openai/human-eval-infilling/blob/88062ff9859c875d04db115b698ed4b0f0395170/data/HumanEval-SingleLineInfilling.jsonl.gz

[10]Nvidia V100 whitepaper: https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

Nvidia V100 model, and N is Nvidia A100. For *Downgrading*, P is A100 and N is V100. Each experiment is as follows. For each pair LLM and inference engine, we search the set of hyperparameters HP_{orig} that maximize throughput on P. Then, we measure throughput of using HP_{orig} on N. After that, we apply hyperparameter optimization on N to find new hyperparameters HP_{best} that maximize throughput on N. Finally, we compare throughput from HP_{best} and HP_{orig} to measure the impact of $\texttt{HF}_{pl}$.

To apply hyperparameter optimization on inference engines, we implement `InfPop` (Inference hyperParameter Optimization). Internally, `InfPop` uses Hyperopt [5], a state-of-the-art hyperparameter optimization, used hyperoptimizing other software engineering tasks [33], [34]. `InfPop` receives as input: *1)* a list of queries to pass to a LLM (in our experiment, queries for code-completion from the *Single-line infilling* benchmark), *2)* a specification of the hyperparameter space. In this experiment, we consider two hyperparameters: number of GPUs and batch size. *3)* An *objective/fitness function*: In this study, the function used by `InfPop` aims at maximizing inference throughput on the mentioned queries. `InfPop` produces as output a set of values for the hyperparameters that maximize the objective. We hyperoptimize `vLLM` and $\texttt{HF}_{pl}$ on each LLM. Given the stochastic nature of Hyperopt, we conduct each experiment 10 times and report the average results.

IV. Results

A. RQ1: What is the shape of the throughput landscape across the hyperparameter space defined by the batch size and the number of GPUs on `vLLM` inference?

Figure 1 shows six plots, each corresponding to an LLM and an inference engine. In Appendix [1] we include all of them, including interactive versions that facilitate the visualization and analysis. Two axes represent the hyperparameter space: batch size and number of GPUs, while the third one -the vertical one- corresponds to the throughput. This analysis does not aim to perform a rigorous classification of these shapes, but to provide evidence though these examples, that the throughput landscapes are not uniform, thus hyperparameter optimization would be useful.

Figure 1a shows the Bloom-1b7 landscape using $\texttt{HF}_{pl}$. It has two peaks where the throughput is much higher (≈ 94.5) than in other parts of the landscape (between 60 and 70 t/s): one uses one GPU and another uses eight GPUs, both with batch size 64. Increasing that increasing the batch size above 64 produces memory errors.

The second case, Fig. 1b, shows the landscape for CodeLlama and $\texttt{HF}_{pl}$. It presents different peaks, most notably: *1)* one single GPU and and batch size 16, *2)* two GPUs and batch size 64, and *3)* four GPUs and batch size 128. This shows that for CodeLlama when one increases the GPUs, the batch size should be also adjusted. This case also shows the challenging of determining optimal hyperparameter values: Codellama-7B using one single GPU can infer batches up to size 16. Putting values of 32 or greater produces memory errors.

The third case, Fig. 1c corresponds to Mistral-7b and $\texttt{HF}_{pl}$. It has a peak at one GPU and batch size 32. Greater batches with one GPU cause memory errors. Beyond this peak, the landscape presents a descending inclined plane as the batch enlarges. The landscape also exhibits high values of throughput (similar to the mentioned peak) with four GPU and small batches (4 and 8). With 4 GPUs, increasing the batch size decreases the throughput.

The fourth case, Fig. 1d corresponds to Gemma-2 and `vLLM`. We observe that: *1)* There is a significant improvement of throughput when using 4 or 8 GPUS compared to using 1 or 2 GPUs. *2)* The landscape has a *plateau*: a vast zone where the values are relatively constant, do not change significantly throughout the plateau, and represent a high-throughput region. We can observe it in yellow. This plateau is located in the region bounded by 4 to 8 GPUs and batch sizes ranging from 128 to 1024. *3)* The throughput given with eight GPUs is slightly higher than that with four. *4)* The highest throughput is with a batch size equal to 258: increasing the size slightly impacts throughput. However, the reduction is more evident when the batch size is smaller than 258.

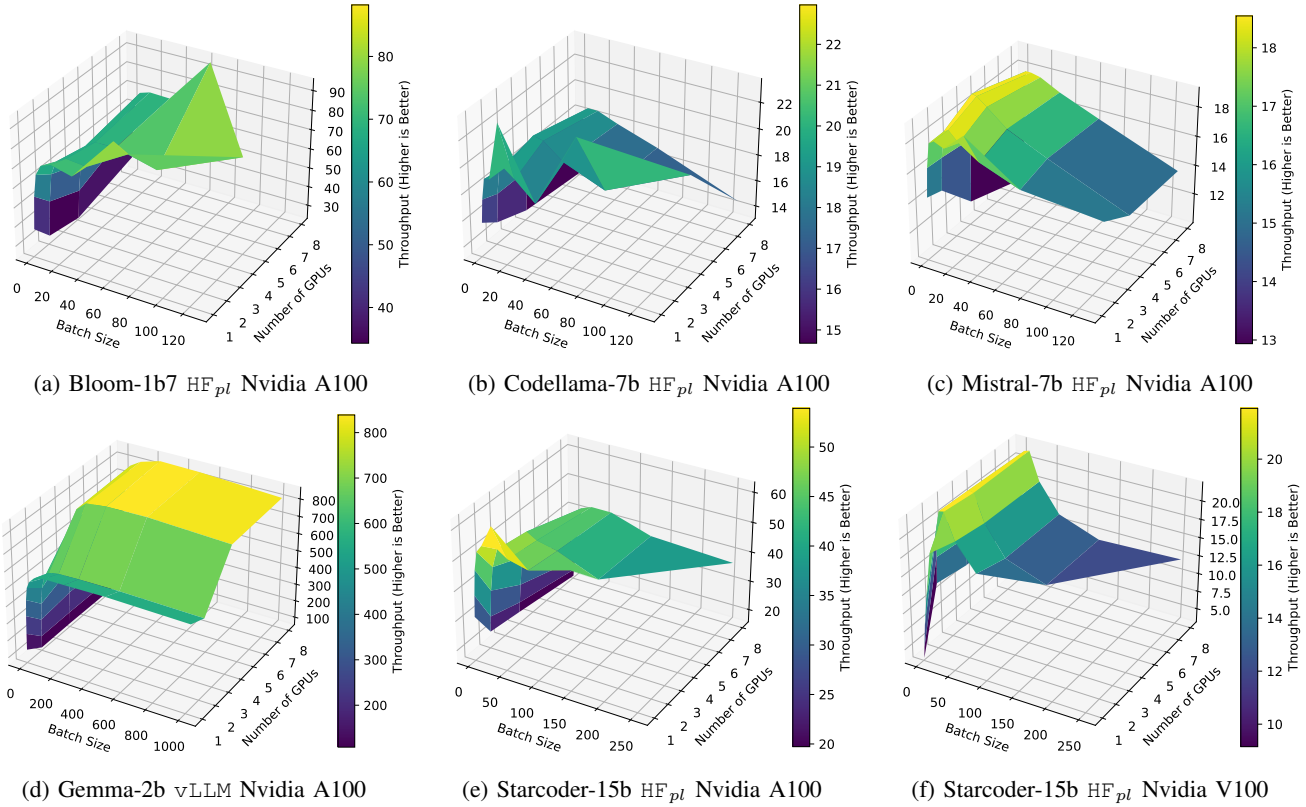
The fifth case, Fig. 1e corresponds to Starcoder-15b and $\texttt{HF}_{pl}$. It has a peak with one GPU and batch size 32 and increasing the batch size to a value just greater than the peak produces a memory error. However, this landscape has two main differences with respect to the two previous examples. First, it has a second peak when using two GPUs and a larger batch size (128). Secondly, the throughput landscape shows a downward slope, indicating that throughput decreases as the number of GPUs increases.

Hardware specifics. The previous landscapes resulted from the execution of inference on a computing node equipped with Nvidia A100. We also inspect the landscape resulted from the V100. Even in the majority of cases the landscapes from both models produces similar shapes (A100 shapes have higher peaks, i.e. higher throughput, because it is a more recent and advanced model), others exhibit more noticeable differences. For example, Fig. 1f shows the execution of StarCoder-15b in a V100 (the Fig at its left shows the landscape of this model from A100). We observe that the shape is different: there is not a peak but a thin and long plateau region bounded by 2 and 8 GPUs batch sizes 8 and 16. This means that, a LLM is deployed in a another GPU model, it might be necessary to adjust the hyperparameters in order to maximize the throughput. In RQ5 we study this.

Visual inspection of the throughput landscape from inference using $\texttt{HF}_{pl}$ and `vLLM` shows irregular landscapes, some with marked peaks. Consequently, to ensure throughput maximization for a particular inference engine and hardware would require an inspection of the landscape.

B. RQ2: For online inference, how does throughput vary with different numbers of GPUs (hardware scaling)?

Table II and Figure 2 show the throughput of online inference performed by $\texttt{HF}_{pl}$ and `vLLM` when scaling Nvidia



(a) Bloom-1b7 $\texttt{HF}_{pl}$ Nvidia A100 (b) Codellama-7b $\texttt{HF}_{pl}$ Nvidia A100 (c) Mistral-7b $\texttt{HF}_{pl}$ Nvidia A100

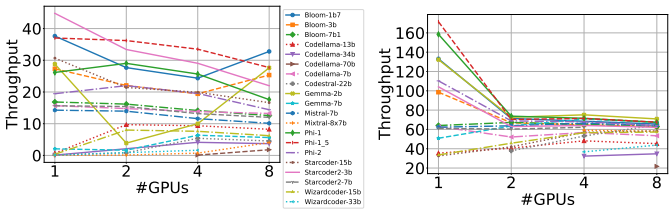
(d) Gemma-2b $\texttt{vLLM}$ Nvidia A100 (e) Starcoder-15b $\texttt{HF}_{pl}$ Nvidia A100 (f) Starcoder-15b $\texttt{HF}_{pl}$ Nvidia V100

Fig. 1: Throughput landscape across the hyperparameter space (batch size and GPUs).

LLM	$\texttt{HF}_{pl}$ (Best)		$\texttt{vLLM}$ (Same #GPUs)		$\texttt{vLLM}$ (Best)		
	GPUs	Thp	Thp	Impr.	GPUs	Thp	Impr.
Bloom-1b7	1	37.6	133.1	3.5X	1	133.1	3.5X
Bloom-3b	1	27.3	98.6	3.6X	1	98.6	3.6X
Bloom-7b1	1	16.8	64	3.8X	2	67.2	4.0X
Codellama-7b	1	15.7	63	4.0X	1	63	4.0X
Codellama-13b	2	9.7	40.6	4.2X	4	48.2	5.0X
Codellama-34b	4	4.1	32.4	7.9X	8	34.7	8.5X
Codellama-70b	8	1.8	21.7	12.1X	8	21.7	12.1X
Codestral-22b	4	5.4	53.2	9.9X	8	64.7	12.0X
Gemma-2b	1	28.8	132.3	4.6X	1	132.3	4.6X
Gemma-7b	4	6.4	70.8	11.1X	4	70.8	11.1X
Mistral-7b	1	14.3	62.3	4.4X	4	66.5	4.7X
Mixtral-8x7b	8	4.1	58	14.1X	4	59.2	14.4X
Phi-1	2	29	73.7	2.5X	1	158.5	5.5X
Phi-1_5	1	37	171.5	4.6X	1	171.5	4.6X
Phi-2	2	22	68.9	3.1X	1	110.3	5.0X
Starcoder-15b	1	30.7	32.5	1.1X	8	58.4	1.9X
Starcoder2-3b	1	44.8	104.1	2.3X	1	104.1	2.3X
Starcoder2-7b	1	15.7	61.1	3.9X	4	62.3	4.0X
Wizardcoder-15b	2	8	45.7	5.7X	8	57.3	7.2X
Wizardcoder-33b	4	1.6	36.8	23.0X	8	43.8	27.4X

TABLE II: Throughput (Thp) from $\texttt{HF}_{pl}$ and $\texttt{vLLM}$ on A100.

A100 GPUs. The table shows: 1) for $\texttt{HF}_{pl}$, the number of GPUs that maximizes the throughput for each model (column Best), 2) for $\texttt{vLLM}$, in column Same, the throughput using that number of GPUs (for a fair comparison), 3) for $\texttt{vLLM}$, the number of GPUs that maximizes the throughput for each



(a) $\texttt{HF}_{pl}$ (b) $\texttt{vLLM}$

Fig. 2: Throughput Variation with Different Numbers of GPUs (Nvidia A100) during Online Inference (batch size = 1).

model (column Best). Tables showing the throughput per GPU, and the results from V100 can be found in our appendix [1].

We observe that the scaling hardware when using HuggingFace has a different effect than when using $\texttt{vLLM}$. All findings also apply for Nvidia V100.

1) **$\texttt{vLLM}$:** From Figure 2b we identify two primary observations. First, scaling from 1 to 2 GPUs using $\texttt{vLLM}$ has a strong (negative) impact on throughput of some models (e.g., Microsoft Phi-1), even for others (such as CodeLlama-13b) has a moderate (positive) impact. Second, scaling from 2 to 4, and then from 4 to 8 GPUs has less impact on throughput: the plot shows that the throughput of most of the model is bounded between 60 and 80 t/s.

We also observe: *a)* with one GPU, throughput presents a large variability in the models: it goes from 32.5 to 171.5

t/s. This variability is reduced after scaling. *b)* 8 models with $\leq$ 3 billion parameters maximize throughput using a single GPU, *c)* when scaling to two GPUs, there are two effects: the first one is that throughput dramatically drops (which is the case for the "smaller" models previously mentioned); the second one is the throughput slinging vary positively in most of models with 13-15 billion parameters. *d)* Models with 7-13 billion parameters maximize throughput using 4 GPUs. *e)* Models with more than 15 billions parameters typicaly maximize throughput with 8 GPUS.

a) **Statistical test for `vLLM`:** Taking into account all GPUs, the ANOVA test rejects the null hypothesis H_0 at the significant level $\alpha = 0.05$ (F-statistic: 5.44, p-value: 0.002) which indicates that there is a statistically significant difference in throughput across different numbers of GPUs for both A100 and V100 GPUs. We suspect that this significant difference is dragged by one of the analyzed groups, GPU 1, which maximizes throughput for some LLM. For that, we performed two additional tests. First, the re-execution of the ANOVA test by just considering trials with 2, 4 and 8 GPUs failed to reject H_0. Secondly, Tukey HSD test (Honestly Significant Difference) reveals significant differences in performance metrics between using GPU 1 and each of the other groups (2, 4, and 8). However, no significant differences were found between GPUs 2, 4, and 8.

2) **$\texttt{HF}_{pl}$:** From Figure 2a, we observe two major differences with respect to `vLLM`: *1)* the throughput are lower than using `vLLM`, independently of the number of GPUs used; *2)* when scaling, there is no dominant pattern evolution of throughput as for `vLLM`. Moreover: *a)* Scaling GPU results in a very slight decrease of throughput from models with 7b parameters (Bloom-7B1, CodeLlama-7b, Mistral-7b). *b)* The decline is more pronounced in StarCoder2-3b. *c)* Scaling cause down and up (e.g., Bloom-1b7, Bloom3-b). *d)* Scaling cause up and down (e.g., Phi-1, Phi-2). In conclusion, using the $\texttt{HF}_{pl}$, scaling generally does not improve throughput for most LLMs (with few exceptions such as Phi-1 and Gemma-7).

a) **Statistical test for $\texttt{HF}_{pl}$:** Based on the results of the ANOVA test, we cannot reject the null hypothesis (H_0) at the 0.05 significance level (F-statistic: 0.37, p-value: 0.77). This means that there is not enough statistical evidence to suggest that the means of the groups -number of GPUs- are significantly different, which can be translated as scaling up GPUs does not produce significant changes in throughput.

3) **Comparison between Inference Engines:** Table II shows the throughput for each model (columns `Thp`) and the improvement in throughput. Taking into account the number of GPUs that led $\texttt{HF}_{pl}$ to maximize throughput (column `vLLM Same`), `vLLM` achieves a mean (median) improvement of $4.3X$ ($6.47X$). However, for 12 LLMs, `vLLM` maximizes throughput by using a different number of GPUs (column `vLLM Best`), achieving a mean (median) improvement of $7.26X$ ($4.80X$) with respect to $\texttt{HF}_{pl}$. In nine of these 12 cases, `vLLM` achieves the gain by using more GPUs and, in the remaining three cases, fewer GPUs. This shows that both `vLLM` and $\texttt{HF}_{pl}$ need to be optimized separately, at least on the number of GPUs, to increase throughput.

The reason behind the difference in throughput is that `vLLM` was designed to achieve fast inference. For that, `vLLM` introduces key improvements. We briefly discuss two of them. First, `vLLM` uses *PagedAttention* [27], a new attention algorithm that manages the attention keys and values from the Transformer architecture [45]. In autoregressive decoding, all the input tokens to the LLM produce their attention key and value tensors, and these tensors are kept in GPU memory to generate the next tokens. This has several problems for LLMs, such as the large amount of memory required and memory waste [26]. PagedAttention proposes a solution inspired by the idea of virtual memory and paging in operating systems: it allows storing continuous keys and values in noncontiguous memory space. The second improvement is *Continuous batching*, initially proposed by [51]. During inference, instead of waiting until every sequence in a batch has completed generation (as traditional static batching does) this approach, once *a sequence* in a batch has completed generation, a new sequence is inserted in its place. This yields a higher resource utilization than static batching [42] and thus increases throughput.

Answer to RQ2: For `vLLM`, GPU scalability benefits larger LLMs ($\approx$13 billion parameters or more), maximizing throughput with the use of eight GPUs. However, scaling up GPUs penalizes smaller models ($\approx$ 3 billion parameters or fewer), decreasing their throughput. For these smaller models, the optimal configuration is using 1 or 2 GPUs. In contrast, using $\texttt{HF}_{pl}$, scaling hardware does not significantly increase throughput for most of the LLMs studied. Additionally, on the same hardware, `vLLM` achieves an average throughput that is 6.47 times higher than that of $\texttt{HF}_{pl}$.

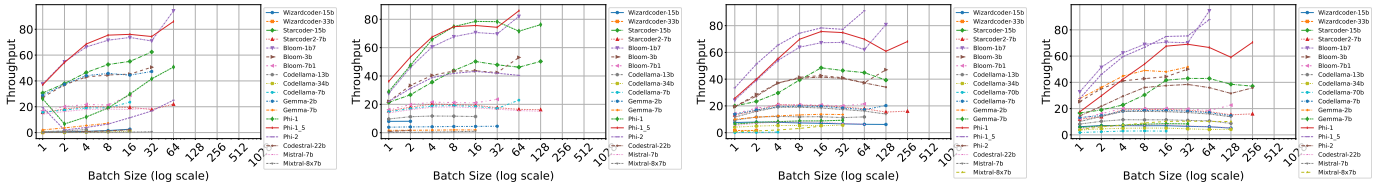
Implications: $\texttt{HF}_{pl}$ may conduct to under-optimal results when the node has more than one GPU (since it uses all GPUs). Furthermore, we recommend hyperoptimizing the number of GPUs to be used during inference.

C. RQ3: For batched inference, how does throughput vary with the batch size?

We study how the throughput varies according to the batch size by using a particular number of GPUs.

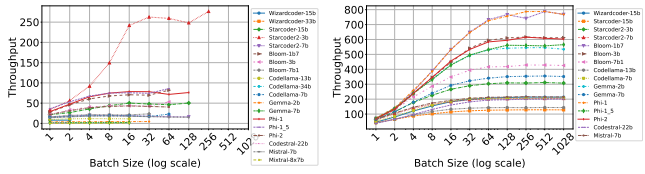
1) **$\texttt{HF}_{pl}$:** Figure 3 shows the throughput using 1, 2, 4 and 8 Nvidia A100 GPUs and $\texttt{HF}_{pl}$.We observe: *a)* The four plots show similar trend. *b)* Scaling on hardware (more GPUs) allow to have larger batches. The reason is that there is more memory on the GPU to allocate bigger batches and thus compute more inputs in parallel. *c)* the maximum throughput is usually reached with the largest batch size that correctly works. For example, Phi-1 achieves the maximum throughput with a batch size 64, larger batches produce memory errors.

a) **Statistical test for $\texttt{HF}_{pl}$:** We compute the ANOVA test for each group of GPU configurations (1, 2, 4, and 8). For both Nvidia A100 and V100, we reject the null hypothesis H_0 for 1, 2 and 4 GPUs, at the significant level $\alpha = 0.05$. All



(a) 1 GPU (b) 2 GPUs (c) 4 GPUs (d) 8 GPUs

Fig. 3: Throughput at difference batch sizes for $\texttt{HF}_{pl}$ using Nvidia A1000 GPUs (outlier Starcoder2-3b removed and shown in Figure 4a)



(a) $\texttt{HF}_{pl}$ (b) vLLM

Fig. 4: Comparison of throughput using two Nvidia A1000 GPUs between $\texttt{HF}_{pl}$ (left) and vLLM (right)

p-values are shown in our appendix. In contrast, we cannot reject H_0 when the trial has eight GPUs.

2) **vLLM:** Figure 4b shows the throughput obtaining with different batch sizes and using two GPUs. The plots for one, four, and eight GPUs exhibit a similar trend and are available in our appendix [1]. From these plots, we observe that: *1)* Throughput generally increases with batch size across most models. *2)* The improvement is more noticeable for smaller to medium batches (up to 128). *3)* Most models achieve their highest throughput around batch sizes of 64 to 256. Beyond this point, throughput tends to plateau, indicating that increasing batch size further does not significantly improve throughput. *4)* For some models, such as Starcoder-15b and Gemma-2b, after the plateau, the throughput shows a slight, but not significant, decrease. *5)* Moreover, for Bloom-1b7 (as well as for other models using 4 or 8 GPUS) the throughput shows a slight decrease followed by a subsequent increase. All these points mean that optimizing the batch size is crucial to maximize throughput in LLM. when using vLLM inference.

a) **Statistical test:** For both types of hardware, Nvidia A100 and V100, and for each number of GPUs, we reject the null hypothesis H_0 at the significant level $\alpha = 0.05$ (p-values listed in the appendix).

3) **Comparison $\texttt{HF}_{pl}$ and vLLM:** Figure 4 shows the throughput from $\texttt{HF}_{pl}$ and vLLM both using two Nvidia A100 GPUs) We observe that: *a)* with batch size 1, in both engines the throughput is below to 100 t/s. *b)* However, throughput from vLLM increases rapidly with batch size. For $\texttt{HF}_{pl}$ the increase is moderate for some models (e.g., Phi-1, Bloom-1b7) or null for others (CodeLlama-7b). *c)* Starcoder-2-3b with $\texttt{HF}_{pl}$ outperforms the other models and has a huge increase from batch size 4. *d)* Since batch size 4, the throughput from vLLMis between 100 and almost 800 t/s. In contrast, for $\texttt{HF}_{pl}$, with the exception of the Starcoder-2-3b mentioned above, throughput never passes 100 t/s. These findings also apply for the inference using 1, 4 and 8 GPUS and GPU Nvidia V100.

a) **Statistical test:** We apply Wilcoxon Signed-Rank Test to test the null hytothesis H_0: *The difference in throughput between* vLLM *and* $\texttt{HF}_{pl}$ *is not significant.* For all number and model of GPUs, the test rejects H_0 at the 0.05 significance level (all p-values are in our appendix). Moreover, we measure the effect size employing Cliff's delta (δ) in order to quantify the magnitude of differences between the throughput from vLLM and $\texttt{HF}_{pl}$. Across all the number and models of GPUs evaluated, the mean δ is 0.8762. This indicates a very large effect size, suggesting a significant difference between the two groups, with vLLM scoring higher than $\texttt{HF}_{pl}$.

Answer to RQ3: In $\texttt{HF}_{pl}$, increasing the batch size leads to slight or moderate increases in throughput for some models (e.g., CodeLlama-7b) and steep increases for others (e.g., Bloom-1b7). Increasing the size of the batch that maximize the throughput often results in out-of-memory errors. Scaling up GPUs allows $\texttt{HF}_{pl}$ to process larger batches. Conversely, increasing vLLM significantly impacts small to medium batches (fewer than 64). For larger batches, changes in throughput are not substantial and do not cause out-of-memory. Overall, vLLM achieves significantly higher throughput than $\texttt{HF}_{pl}$ on the two hardware devices tested.

Implications: Optimizing batch size is crucial to maximize throughput in LLM. In $\texttt{HF}_{pl}$, determining the optimal batch size and number of GPUs to achieve maximum throughput is challenging because is bonded by the GPU memory. This insight is valuable for hyperparameter optimization, especially in scenarios where throughput is a critical performance metric.

D. RQ4: How does throughput vary between different GPU models during inference?

For vLLM, the increase of throughput using A100 compared to V100 is on average 83.5% (median 77.6). The models that exhibit the largest difference are Gemma-7b, Codellama-13b, 34b and 70b, Wizardcoder-15b, all with 100% or greater improvement by using A100. Appendix [1] includes a detailed table with the improvement for each model and hardware.

We compute Wilcoxon Signed-Rank Test to test the null hypothesis H_0: *The difference in throughput between Nvidia*

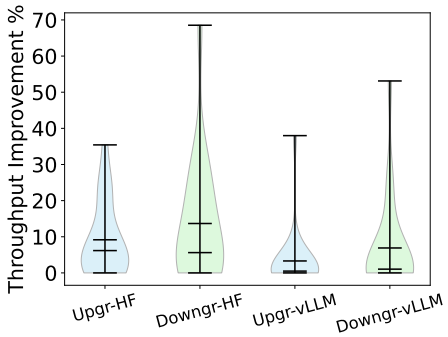


Fig. 5: Distribution of throughput improvement for $\texttt{HF}_{pl}$ given by hyperparameter optimization in hardware upgrading (from Nvidia V100 to A100) and downgrading (from A100 to V100).

A100 and V100 is not significant for `vLLM`. We reject H_0 at the significant level $\alpha = 0.05$ (p-value = 2.01). We measure the effect size to quantify the differences of throughput between A100 and V100 using Cliff's delta [9]. A delta value of 0.406 suggests a moderate effect size, indicating a moderate difference between A100 and V100 using `vLLM`.

For $\texttt{HF}_{pl}$, the increase of throughput using A100 is lower on average, 47% (median 23.3%). Notally, the increase is greater for the largest models such as WizardCoder-33b (535.6%) and Codellama-70b-hf (276%), but smaller for the small models such as Gemma-2b (1.9%), Phi-2 (21.8%). We reject H_0 for $\texttt{HF}_{pl}$ at the significant level $\alpha = 0.05$ (p-value = 3.88). The measurement of effect size using Cliff's delta is 0.1333, which indicates a small but noticeable effect accross throughput from A100 and V100.

Answer to RQ4: Upgrading (or downgrading) the GPU model has a significant impact on the throughput of `vLLM`, with an improvement of 83.5% on average) and on $\texttt{HF}_{pl}$ with an improvement of 47% on average.

Implications: Developers of AI-based systems need to analyze the following trade-off, either: *a)* Maximizing Throughput: Opt for the Nvidia A100 to achieve maximum throughput. This choice involves higher costs due to the premium price of state-of-the-art hardware; or *b)* Reducing Operational Costs: Choose *older* hardware like Nvidia V100, which might result in lower throughput but eventually reduce operational expenses. This option is particularly advantageous for LLMs where the difference in throughput between A100 and V100 is modest (e.g., $\texttt{HF}_{pl}$ and small LLMs such as Microsoft-Phi).

E. RQ5: To what extent does applying Hyperparameter Optimization improve throughput in LLM?

Fig 5 shows the distribution of improvement in throughput given by the new hyperparameter values found by our tool `InfPop`, in the context of GPU upgrade (from Nvidia V100 to A100) and downgrade (from A100 to V100). For upgrade, optimization produces $\texttt{HF}_{pl}$ a median increase (avg) in throughput of 6.4% (9.16%). For some models, e.g., Starcoder-15b the improvement is higher (24%), while for others three there is no improvement (i.e., the best configuration, in terms of throughput, of $\texttt{HF}_{pl}$ on V100 is also the best on A100). For downgrade, the median improvements are 5.61% (average 13.7%). For `vLLM` the improvements are lower, in both upgrading (median 3.3%) and downgrading (median 6.9%). The reason is that, as shown in Figure 4, optimizing the evaluated hyperparameters in `vLLM` has a lesser impact on throughput than in $\texttt{HF}_{pl}$.

Answer to RQ5: Our results suggest that applying hyperparameter optimization during changes in the inference infrastructure —such as upgrades or downgrades of GPU devices— can yield improvements in throughput. On average, hyperparameter optimization enhances throughput from $\texttt{HF}_{pl}$ by 9.16% during GPU upgrades and 13.7% during GPU downgrades.

Implications: Applying hyperparameter optimization during hardware upgrades (e.g., GPU) is crucial to maximize the benefits provided by the new hardware. Applying hyperparameter optimization during hardware downgrades is essential because the transition from a newer model (A100) to an older one (V100) typically results in a decrease in throughput. This study demonstrates that hyperparameter optimization can help to mitigate these performance losses.

V. Discussion

A. Deployment of LLMs on available hardware

Both `vLLM` and $\texttt{HF}_{pl}$ inference engines are able to deploy the largest LLM using the CPU and just one GPU. The reason is that these engines provide specially mechanisms, such as HF Accelerate [19] activated via the hyperparameter `device_map="auto"`, to determine automatically where to put each layer of the model depending on the available resources. In the case of Accelerate, it first uses the maximum space available on the GPU(s), and when it still needs space, it stores the remaining weights on the CPU, RAM, and finally on the hard drive as memory-mapped tensors. For example, Starcoder2-3b and CodeLlama-7b fully fit in one GPU (either A100 or V100). From Table II we observe using just 1 GPU maximizes their throughput. For them, if one uses $\texttt{HF}_{pl}$ with `device_map="auto"` on a infrastructure with more than one A100 GPU, the throughput would be negatively affected. The other models can perform the inference with 1 GPU, but also needs to store weights on CPUs and eventually on disk (e.g., Mixtral), damaging the throughput.

The amount of GPU memory has an impact. For example, $\texttt{HF}_{pl}$ achieves to store all CodeLlama-70b weights in eight A100s, each with 40Gb. However, it cannot do that on the eight V100 with 32GB, which requires the storage of some parameters in the CPU. This has an impact on throughput (1.79 vs 0.21), beyond the technological improvements introduced by the Nvidia A100.

B. Correlation between throughput, HumanEval score and number of parameters

Table II shows that, for both $\texttt{HF}_{pl}$ and vLLM, those that maximize throughput are the models with smaller number of parameters, such as Bloom-1b7, Gemma-2, and Starcoder2-3b. The Pearson correlation between parameters and throughput (columns $\texttt{HF}_{pl}$ Best, vLLM Same and vLLM Best) returns coefficients between -0.623 and -0.67, suggesting a noticeable inverse relationship between throughput and the number of parameters (p-values between 0.001 and 0.003).

The largest models generally achieve the highest performance scores in HumanEval, as shown in Table I. The correlation coefficient between HumanEval and the parameters is 0.527 (p-value 0.017), indicating a moderate positive relationship. This suggests that models with more parameters tend to have higher HumanEval scores.

We also compare the correlation between throughput and HumanEval. For $\texttt{HF}_{pl}$ the coefficient is -0.58 (p-value 0.0072). As throughput increases, HumanEval tends to decrease. For vLLM, considering the 'best' throughput (Table II) the coefficient is -0.35 (p-value 0.122): As throughput increases, HumanEval tends to decrease slightly. At the significant level $\alpha = 0.05$, the correlation is not significant. These results suggest that vLLM achieves a more favorable trade-off between performance and throughput compared to $\texttt{HF}_{pl}$.

C. Threats to Validity

It could be the risk that HumanEval is not representative of the code completion task. Neverteless, we consider it because it has been used on similar tasks such as infilling [13], [4].

There are other inference engines that could eventually achieve state-of-the-art throughput, such as TensorRT-LLM [38]. We choose vLLM over other efficient engines because it first introduced a novel mechanism (PagedAttention [27]), later adopted by other engines such as TensorRT-LLM.

VI. Related work

Wang et al. [46] presented EcoOptiGen, a hyperparameter optimization for LLM inference. Their and our work have substantial differences. They hyperoptimize the inference of closed LLM from OpenAI, accessible through an API for optimizing accuracy. In contrast, our work optimizes on throughput by exploring parameters such as number of GPUs, batch sizes, which are not accessible via the OpenAI API.

Previous work has study inference issues. Gao et al. [14] studied low GPU utilization issues from jobs submitted to Platform-X, a Microsoft internal deep learning platform. They classified 706 issues discovered across 400 deep learning jobs, and found that the 25.64% of the issues were related to "Improper Batch Size" and the 3.12% are related to "Insufficient GPU Memory". Zhang et al. [53] conducted a similar study another deep learning platform in Microsoft (Philly) and found that 8.8% of the issues were related to GPU out-of-memory. These results show that InfPop (after changing the fitness function) could be useful to avoid these types of issues. Motivated by the out of memory failures, tools such as DNNMem [16] and DNNPerf [15] were proposed for predicting GPU memory consumption and performance of deep learning models. Similarly, Cao et al. [6] studied performance problems of TensorFLow and Keras frameworks from StackOverflow posts. They found 12 performance problems related to "Improper Hyper Parameter". In future work, InfPop could be adapted to try to fix them.

Previous work has also focused on studying training and/or inference configuration via hyperparameters. McCandlish et al. [35] predicted the largest useful batch size for neural network training, addressing the trade-off between training speed and computational efficiency. In our paper, we focus on inference. Gao et al. [17] proposed DnnSAT, a resource-guided approach for deep learning models to help existing AutoML tools [21] (such as Hyperparameter Optimization) efficiently reduce the configuration space. DnnSAT could be integrated into our tool InfPop to reduce the number of configurations.

Previous work has conducted related empirical work on LLM inferences. Samsi et al. [40] benchmarked the energy consumed by Llama models: 7b, 13b and 65B. Beyond the research goals, there are some differences with our work, notably the model evaluated (3 vs. 20) and the inference libraries (Llama scripts [11], based on Pytorch and FairScale [12], vs $\texttt{HF}_{pl}$ and vLLM). Xhou et al. [55] evaluated the speed-ups achieved by employing quantization techniques on two models LLaMA-2-7B and 13B. Argerich et al. [3] presented EnergyMeter, a software-based tool to measure the energy consumption of different hardware components. For evaluation, they considered LLMs with at most 7b parameters (we consider LLMs up to 70b) and use HuggingFace Transformers for inference. Coignion et al. [10] evaluated the code generated by 18 LLMs on the Leetcode benchmark. The main differences from our work, beyond the goal, are the LLMs evaluated (they consider smaller LLMs, up to 15b parameters) and the inference engine used (DeepSpeed vs $\texttt{HF}_{pl}$and vLLM). Yarally et al. [50] studied how batch inference affects the energy consumption of computer vision tasks using Pytorch. Alizadeh et al. [2] evaluated the energy consumption of deep learning runtime infrastructures such as ONNX, PyTorch and TensorFlow. Beyond differences in research goals, these two works do not focus on LLMs.

VII. Conclusions and Future work

In this paper, we conduct a large-scale evaluation of the throughput of 20 LLMs using two inference engines. We inspect the throughput landscape from the hyperparameter space and demonstrate that hyperparameter optimization can be applied to improve throughput.

In future work, we plan to: *1)* Evaluate the performance of other inference engines such as those listed in the surveys [36], [55]. *2)* Analyze the landscapes from other measures such as resource utilization (CPU, GPU) or latency. *3)* Incorporate these measures into InfPop's optimization objective. *4)* Evaluate other hyperparameter optimization approaches and space reduction techniques such as [6] *5)* Study and optimize

others hyperparameters that may affect throughput and the eventual new measures. *6)* Incorporate the specification of these hyperparameter spaces into `InfPop`. *7)* Release a usable version of `InfPop` and present its architecture, design, implementation decisions, and usage in a demo/tool track paper.

REFERENCES

[1] Appendix.https://anonymous.4open.science/r/infpop-0223/, 2024.

[2] Negar Alizadeh and Fernando Castor. Green ai: A preliminary empirical study on energy consumption in dl models across different runtime infrastructures. *arXiv preprint arXiv:2402.13640*, 2024.

[3] Mauricio Fadel Argerich and Marta Patiño-Martínez. Measuring and improving the energy efficiency of large language models inference. *IEEE Access*, 12:80194–80207, 2024.

[4] Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle, 2022.

[5] James Bergstra, Daniel Yamins, and David Cox. Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures. In Sanjoy Dasgupta and David McAllester, editors, *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pages 115–123, Atlanta, Georgia, USA, 17–19 Jun 2013. PMLR.

[6] Junming Cao, Bihuan Chen, Chao Sun, Longjie Hu, Shuaihong Wu, and Xin Peng. Understanding performance problems in deep learning systems. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 357–369, 2022.

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.

[8] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. Nvidia a100 tensor core gpu: Performance and innovation. *IEEE Micro*, 41(2):29–35, 2021.

[9] Norman Cliff. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological bulletin*, 114(3):494, 1993.

[10] Tristan Coignion, Clément Quinton, and Romain Rouvoy. A Performance Study of LLM-Generated Code on Leetcode. In *28th International Conference on Evaluation and Assessment in Software Engineering (EASE'24)*, Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE'24), Salerno, Italy, June 2024.

[11] Facebook Research,. Llama. https://github.com/facebookresearch/llama, 2023.

[12] FairScale authors. Fairscale: A general purpose modular pytorch library for high performance and large scale training. https://github.com/facebookresearch/fairscale, 2021.

[13] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999*, 2022.

[14] Y. Gao, Y. He, X. Li, B. Zhao, H. Lin, Y. Liang, J. Zhong, H. Zhang, J. Wang, Y. Zeng, K. Gui, J. Tong, and M. Yang. An empirical study on low gpu utilization of deep learning jobs. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 880–880, Los Alamitos, CA, USA, apr 2024. IEEE Computer Society.

[15] Yanjie Gao, Xianyu Gu, Hongyu Zhang, Haoxiang Lin, and Mao Yang. Runtime performance prediction for deep learning models with graph neural network. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 368–380, 2023.

[16] Yanjie Gao, Yu Liu, Hongyu Zhang, Zhengxian Li, Yonghao Zhu, Haoxiang Lin, and Mao Yang. Estimating gpu memory consumption of deep learning models. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, page 1342–1352, New York, NY, USA, 2020. Association for Computing Machinery.

[17] Yanjie Gao, Yonghao Zhu, Hongyu Zhang, Haoxiang Lin, and Mao Yang. Resource-guided configuration space reduction for deep learning models. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 175–187. IEEE, 2021.

[18] Hadi Ghaemi, Zakieh Alizadehsani, Amin Shahraki, and Juan M Corchado. Transformers in source code generation: A comprehensive survey. *Journal of Systems Architecture*, page 103193, 2024.

[19] Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. Accelerate: Training and inference at scale made simple, efficient and adaptable. https://github.com/huggingface/accelerate, 2022.

[20] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.

[21] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.

[22] Mojan Javaheripi, Sébastien Bubeck, Marah Abdin, Jyoti Aneja, Sebastien Bubeck, Caio César Teodoro Mendes, Weizhu Chen, Allie Del Giorno, Ronen Eldan, Sivakanth Gopi, et al. Phi-2: The surprising power of small language models. *Microsoft Research Blog*, 2023.

[23] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

[24] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[25] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*, 2024.

[26] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Yu, Joey Gonzalez, Hao Zhang, and Ion Stoica. vllm: Easy, fast, and cheap llm serving with pagedattention. https://blog.vllm.ai/2023/06/20/vllm.html, 2023.

[27] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[28] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. Llm inference serving: Survey of recent advances and opportunities. *arXiv preprint arXiv:2407.12391*, 2024.

[29] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

[30] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. Textbooks are all you need ii: **phi-1.5** technical report. *arXiv preprint arXiv:2309.05463*, 2023.

[31] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.

[32] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*, 2023.

[33] Andre Lustosa and Tim Menzies. Optimizing predictions for very small data sets: a case study on open-source project health prediction. arXiv:2301.06577, 2023.

[34] Matias Martinez, Jean-Rémy Falleri, and Martin Monperrus. Hyperparameter optimization for ast differencing. *IEEE Transactions on Software Engineering*, 49(10):4814–4828, 2023.

[35] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162*, 2018.

[36] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. Towards efficient generative large language model serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234*, 2023.

[37] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. *ICLR*, 2023.

[38] Nvidia. Tensorrt-llm. https://github.com/NVIDIA/TensorRT-LLM, 2023.

[39] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[40] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devesh Tiwari, and Vijay Gadepally. From words to watts: Benchmarking the energy costs of large language model inference. In *2023 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–9, 2023.

[41] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. Intellicode compose: code generation using transformer. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, page 1433–1443, New York, NY, USA, 2020. Association for Computing Machinery.

[42] Eric Liang Sy Cade Daniel, Chen Shen and Richard Liaw. How continuous batching enables 23x throughput in llm inference while reducing p50 latency. https://www.anyscale.com/blog/continuous-batching-llm-inference, 2023.

[43] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.

[44] MosaicML NLP Team. Introducing mpt-7b: A new standard for open-source, commercially usable llms, 2023. Accessed: 2023-05-05.

[45] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[46] Chi Wang, Xueqing Liu, and Ahmed Hassan Awadallah. Cost-effective hyperparameter optimization for large language model generation inference. In Aleksandra Faust, Roman Garnett, Colin White, Frank Hutter, and Jacob R. Gardner, editors, *Proceedings of the Second International Conference on Automated Machine Learning*, volume 228 of *Proceedings of Machine Learning Research*, pages 21/1–17. PMLR, 12–15 Nov 2023.

[47] Ryen W. White. Navigating complex search tasks with ai copilots, 2023.

[48] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In Qun Liu and David Schlangen, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics.

[49] BigScience Workshop, Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, et al. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.

[50] Tim Yarally, Luís Cruz, Daniel Feitosa, June Sallou, and Arie van Deursen. Batching for green ai - an exploratory study on inference. In *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 112–119, 2023.

[51] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA, July 2022. USENIX Association.

[52] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. A survey on large language models for software engineering. *arXiv preprint arXiv:2312.15223*, 2023.

[53] Ru Zhang, Wencong Xiao, Hongyu Zhang, Yu Liu, Haoxiang Lin, and Mao Yang. An empirical study on program failures of deep learning jobs. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20, page 1159–1170, New York, NY, USA, 2020. Association for Computing Machinery.

[54] Ziyin Zhang, Chaoyu Chen, Bingchang Liu, Cong Liao, Zi Gong, Hang Yu, Jianguo Li, and Rui Wang. Unifying the perspectives of nlp and software engineering: A survey on language models for code. *arXiv preprint arXiv:2311.07989*, 2023.

[55] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, et al. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294*, 2024.

[56] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, MAPS 2022, page 21–29, New York, NY, USA, 2022. Association for Computing Machinery.

[57] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. Measuring github copilot's impact on productivity. *Commun. ACM*, 67(3):54–63, feb 2024.